

Guia 100% Mastigado: publicar o EsporteID no iPhone com push funcionando

Este guia já está preenchido com seus dados: **EsporteID**, bundle **com.esporteid.app**, suporte em <https://esporteid.com.br/suporte> e privacidade em <https://esporteid.com.br/privacidade>.

Importante: você não precisa comprar um Mac, mas vai precisar de um **Mac remoto** ou alguém com Mac para fechar o push do iPhone.

PARTE 1 — O que já foi preparado

- suporte público criado
- privacidade pública criada
- termos públicos criados
- app iOS simplificado para iPhone only
- app iOS simplificado para portrait only
- backend preparado para aceitar token nativo do iPhone
- dispatcher preparado para payload iOS via FCM
- AppDelegate preparado para Firebase Messaging
- UI de push pronta para iPhone

PARTE 2 — O que você precisa ter

1. Conta Apple Developer
2. Conta App Store Connect
3. Projeto Firebase do Android
4. Mac remoto
5. iPhone real

PARTE 3 — Dados oficiais

- **App Name:** EsporteID
 - **Bundle ID:** com.esporteid.app
 - **Website:** <https://esporteid.com.br>
 - **Support URL:** <https://esporteid.com.br/suporte>
 - **Privacy Policy URL:** <https://esporteid.com.br/privacidade>
-

PARTE 4 — Publicar o site antes do app

1. <https://esporteid.com.br/suporte>
 2. <https://esporteid.com.br/privacidade>
 3. <https://esporteid.com.br/termos>
-

PARTE 5 — Criar o app no App Store Connect

1. criar novo app
 2. nome: EsporteID
 3. plataforma: iOS
 4. idioma: Português (Brasil)
 5. Bundle ID: com.esporteid.app
-

PARTE 6 — Preparar o Firebase para iPhone

1. abrir o Firebase Console
 2. adicionar app iOS
 3. bundle: com.esporteid.app
 4. baixar GoogleService-Info.plist
-

PARTE 7 — Preparar APNs na Apple

1. abrir Apple Developer
2. abrir o App ID com.esporteid.app
3. habilitar Push Notifications

4. criar chave APNs .p8

Guarde:

- Key ID
- Team ID
- arquivo .p8

PARTE 8 — Ligar APNs no Firebase

No Firebase Cloud Messaging do app iOS, subir a chave .p8 e preencher:

- Key ID
- Team ID
- Bundle ID = com.esporteid.app

PARTE 9 — Abrir o projeto no Mac remoto

```
npm install
npm run cap:sync:ios
npm run cap:open:ios
```

PARTE 10 — O que fazer no Xcode

1. adicionar GoogleService-Info.plist ao target
2. abrir Signing & Capabilities
3. selecionar sua conta Apple
4. confirmar Push Notifications
5. garantir FirebaseCore e FirebaseMessaging
6. confirmar bundle com.esporteid.app

PARTE 11 — Gerar o primeiro build

Primeiro mande para **TestFlight**.

PARTE 12 — Testar o push no iPhone real

1. instalar no iPhone
 2. abrir e logar
 3. ativar o push
 4. aceitar a permissão
 5. testar recebimento
 6. testar app aberto, em background e fechado
-

PARTE 13 — O que testar além do push

- login
 - cadastro
 - suporte
 - termos
 - privacidade
 - câmera/foto
 - localização com app aberto
 - compartilhamento
-

PARTE 14 — O que preencher na App Store

- nome do app
 - descrição
 - categoria
 - classificação etária
 - screenshots de iPhone
 - Support URL: <https://esporteid.com.br/suporte>
 - Privacy Policy URL: <https://esporteid.com.br/privacidade>
-

PARTE 15 — Review Notes

O app EsporteID permite cadastro, perfil esportivo, acompanhamento de partidas
Suporte público: <https://esporteid.com.br/suporte>

Política de privacidade: <https://esporteid.com.br/privacidade>

Exclusão de conta e pedidos LGPD ficam disponíveis para usuários autenticados

PARTE 16 — Fluxo certo de publicação

1. publicar o site
2. criar app iOS no Firebase
3. baixar GoogleService-Info.plist
4. criar chave APNs na Apple
5. subir APNs no Firebase
6. abrir projeto no Mac remoto
7. adicionar GoogleService-Info.plist no Xcode
8. confirmar Signing & Capabilities
9. gerar build
10. mandar para TestFlight
11. testar no iPhone
12. corrigir se necessário
13. enviar para App Review
14. publicar

PARTE 17 — Checklist final

- ☐ Conta Apple Developer ativa
- ☐ App criado no App Store Connect
- ☐ Bundle ID correto: com.esporteid.app
- ☐ Support URL funcionando
- ☐ Privacy Policy URL funcionando
- ☐ Projeto Firebase do Android localizado
- ☐ App iOS criado no Firebase
- ☐ GoogleService-Info.plist baixado
- ☐ Push Notifications ativado no Apple Developer
- ☐ Chave APNs .p8 criada
- ☐ Key ID anotado
- ☐ Team ID anotado

- ☐ APNs configurado no Firebase
- ☐ Projeto abriu no Xcode
- ☐ GoogleService-Info.plist adicionado ao target iOS
- ☐ Signing configurado
- ☐ Capability Push Notifications confirmada
- ☐ Build enviado para TestFlight
- ☐ Testado em iPhone real
- ☐ Push funcionando com app aberto
- ☐ Push funcionando com app em segundo plano
- ☐ Push funcionando ao tocar na notificação
- ☐ Screenshots prontas
- ☐ Review Notes preenchidas
- ☐ Login de teste separado, se necessário
- ☐ Enviado para revisão

PARTE 18 — Se der erro

- **Erro de signing:** conta Apple / team / provisioning
- **Erro de push iPhone:** APNs, Firebase, GoogleService-Info.plist ou FirebaseMessaging
- **Android funciona e iPhone não:** normalmente é problema Apple/Firebase, não do backend Android

Resumo final

O Android já estava pronto. Para o iPhone funcionar com push, você precisa ligar corretamente **Apple + Firebase + Xcode**, testar no **iPhone real** e só depois publicar.